SQL
S6

EDC- 2026
Sh. Dowlatshahi

Session 6 :Advanced Queries & practice

- Overview of advanced SQL concepts with practical examples, including subqueries, CASE, expressions, window functions, and other techniques for data analysis and query optimization.

Course content

S1: Introduction to Databases & SQL

S2: DDL

S3: DML

S4: Aggregation and Basic Analysis

S5: Working with Multiple Tables

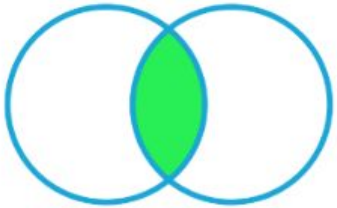
S6: Advanced Queries

S7: Final presentation & Exam

Recap:

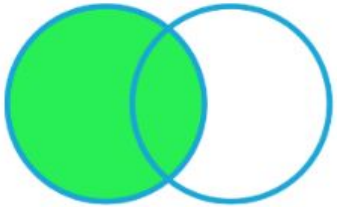
A JOIN in SQL is used to combine rows from two or more tables based on a **related/ matching column** between them.

Inner Join



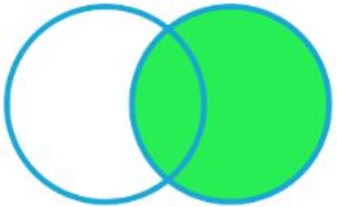
Returns matched rows only

Left Join



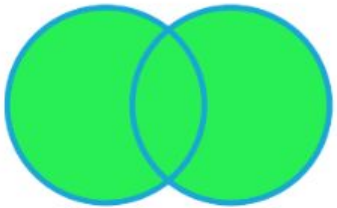
Returns matched rows and left table's rows

Right Join



Returns matched rows and right table's rows

Full Join



Returns matched rows and both tables' rows

What is a Subquery?

- A subquery (inner or nested query) is a query inside another query.
  used to provide input or filter the main query.
- Subquery **Runs first** to give data to main query.
- It is Useful for complex data filtering.
- It helps you write complex queries by breaking them into smaller parts.

How Subquery Works?

- The subquery runs first and returns a result.
- The main query **uses this result** to get the final data.
- Subqueries are usually inside parentheses ().

Where to use Subquery?

- You need a calculated value (like total count or max) for each row.
- You want to filter data based on results from another table.
- The logic is easier to understand when separated.

Case Study: Products

Tables: Products and Product_groups

```
CREATE TABLE Products (
product_id VARCHAR(3),
product_name VARCHAR(20),
price NUMERIC,
group_id VARCHAR(1)
);

SELECT * FROM Products;
```

```
CREATE TABLE Product_groups(
group_id VARCHAR(1),
group_name VARCHAR(20)
);

SELECT * FROM Product_groups;
```

Table 1: Products

```
CREATE TABLE Products (
product_id VARCHAR(3),
product_name VARCHAR(50),
price NUMERIC,
group_id VARCHAR(1)
);

SELECT * FROM Products;
```

```
COPY Products (
product_id,
product_name,
price,
group_id
)
FROM
'C:\Users\user\Downloads\products.csv'
DELIMITER ','
CSV HEADER
ENCODING 'UTF8';
```

Table 2:Product_groups

```
CREATE TABLE Product_groups(
group_id VARCHAR(1),
group_name VARCHAR(20)
);

SELECT * FROM Product_groups;
```

```
COPY Product_groups(
group_id,
group_name
)
FROM
'C:\Users\user\Downloads\product_groups.csv'
DELIMITER ','
CSV HEADER
ENCODING 'UTF8';
```

Example:

Find all products where group_name is “Smartphone”

Example:

Find products where group_name = 'Smartphone'

Step 1: Get the group_id for 'Smartphone'

```
SELECT group_id
FROM product_groups
WHERE group_name = 'Smartphone';
```

Example:

Find products where group_name = 'Smartphone'

Step 2: Use that group_id in the products query

```
SELECT product_name
FROM Products
WHERE group_id = '1';
```

Example with 2 Steps

Find products where group_name = 'Smartphone'

Step 1: Get the group_id for 'Smartphone'

```
SELECT group_id
FROM product_groups
WHERE group_name = 'Smartphone';
```

Step 2: Use that group_id in the products query

```
SELECT product_name
FROM products
WHERE group_id = 1;
```

But this approach requires running two queries.
Instead, you can simplify it with a **subquery.**

Same Example with Subquery

```
SELECT product_name
FROM products
WHERE group_id = (
  SELECT group_id
  FROM product_groups
  WHERE group_name = 'Smartphone'
);
```

Same Example with Subquery

```
SELECT product_name
FROM products
WHERE group_id = (
  SELECT group_id
  FROM product_groups
  WHERE group_name = 'Smartphone'
)
ORDER BY product_name DESC;
```

- The subquery finds the group_id of 'Smartphone'.
- The outer query selects product names in that group and sorts them by name.

Same Example with Join

```
SELECT products.product_name, product_groups.group_name
FROM products LEFT JOIN product_groups
ON product_groups.group_id= products.group_id
WHERE group_name='Smartphone';
```

Subquery

Subquery used to provide values or intermediate results to the main (outer) query.

In PostgreSQL, subqueries can appear in different places, like:

- In the SELECT clause
- In the FROM clause
- In the WHERE clause

Subquery in the SELECT clause

Use this to calculate a value for each row.

```
SELECT
group_name,  (SELECT COUNT(*)  FROM products WHERE
products.group_id = product_groups.group_id) AS product_count

FROM product_groups;
```

For each product_group, the subquery counts how many products belong to that group (product count).

The result shows the group name and the number of products in it

Subquery in the WHERE clause

Use this to filter rows based on another query.

```
SELECT product_name
FROM products
WHERE group_id IN (
SELECT group_id
FROM product_groups
WHERE group_name LIKE '%Smart%'
);
```

The subquery selects group_ids where the group_name contains 'Smart'

The outer query returns all products belonging to those groups

Subquery in the FROM clause

Use this to treat a subquery as a temporary table.

```
SELECT group_name, AVG(price) avg_price
FROM (
SELECT pg.group_name, p.price
FROM products AS p
JOIN product_groups AS pg ON p.group_id = pg.group_id
WHERE p.price > 500
) AS expensive_products
GROUP BY group_name;
```

The inner query selects products with price > 500 and their group names.

The outer query groups by product group and calculates the average price of these expensive products.

Alias

Table:
FROM products **AS** p
FROM products p

Column:
SELECT AVG(price) **AS** average_price
SELECT AVG(price) average_price

Using **AS** makes queries clearer and easier to read.

Same example with aggregation and join

```
SELECT COUNT(product_name) AS Product_count, group_name
FROM products LEFT JOIN product_groups
ON products.group_id=product_groups.group_id
GROUP BY (product_groups.group_name);
```

How to round a decimal number to n decimal places in PostgreSQL?

```
SELECT ROUND(column_name, 2)
FROM table_name;
```

n=2

3.65809808798709 -> 3.66

This works with NUMERIC, DECIMAL, and FLOAT values.

Joins VS. Subqueries

- JOINs are used to merge related tables to get more columns.
- Subqueries are used to ask a query inside another query.
- JOINs give a flat result: multiple tables combined into one row per match.
- Subqueries allow nesting: one query depends on the result of another.
- JOINs are usually faster and better for large datasets.
- Subqueries can be simpler when filtering, counting, or comparing values

Choosing between them depends on:

- What you’re trying to achieve.
- Which version is more readable.
- How performance is affected.

1. What you’re trying to achieve:

- Sometimes you want to filter rows based on another table → a **subquery** might be easier.

- Sometimes you want to combine data from two tables side by side → a **JOIN** works better.

2. Which version is more **readable**:

. If the query is easier for you or others to understand, go with that option.

- For example, some people find JOINs clearer for combining tables, while others prefer subqueries for filtering.

3. How performance is affected:

- Depending on the database and query, sometimes JOINs run faster.
- Other times, subqueries can be more efficient.
- It’s good to test both if performance matters.

Performance Check

Write each of these phrases **before your query:**

- **EXPLAIN:**
It shows the estimated cost, rows, and plan PostgreSQL thinks it will use but does not execute the query.

- **EXPLAIN ANALYZE:**
This is the most accurate way to analyze performance. It tells you:

- Actual time taken for each step
- Rows returned at each stage
- Whether PostgreSQL used an index or a sequential scan
- Total execution time

expression IN (subquery)

```
SELECT name FROM employees
WHERE department_id IN (SELECT id FROM departments WHERE
location = 'Paris');
```

Returns TRUE if expression matches any row from the subquery.

(EXISTS): Think of EXISTS as asking, “Does any row exist in the subquery that meets the condition?” As soon as one row is found, it stops checking—making it efficient.

(IN): Useful when checking if a value appears in a list. It’s like saying, “Is this value among the results returned by the subquery?”

(NOT IN): Similar to IN, but inverted. A hidden trap is that if the subquery returns NULL, the whole comparison may become unknown (NULL), and the row might be excluded unexpectedly.

(ANY / SOME): Use when you want to compare your value to at least one from the subquery. For example, if you want prices lower than at least one competitor, use < ANY.

(ALL): Use when you want to compare your value to every value returned by the subquery. For example, “lower than all competitors” is a strong condition.

(Single-Row Comparison): This is a compact way to match against multiple columns at once, but only works when the subquery returns exactly one row — not zero, not more.

expression NOT IN (subquery)

Returns TRUE if expression matches none of the subquery results.
Careful with NULLs, result could be NULL.

Example- expression NOT IN (subquery)

```
SELECT product_name
FROM products
WHERE group_id NOT IN (
  SELECT group_id
  FROM product_groups
  WHERE group_name IN ('Smartphone', 'Laptop')
);
```

The subquery finds group IDs for 'Smartphone' and 'Laptop'.

The outer query selects products whose group_id is not in those groups.

This way you filter out all products from those two groups.

Window Function -example

Example table: `sales`

seller	day	amount
Alice	2024-01-01	100
Alice	2024-01-02	150
Bob	2024-01-01	200
Bob	2024-01-02	50

Example: Sales table

```
CREATE TABLE sales (
    seller VARCHAR(50),
    day DATE,
    amount INT
);
```

```
INSERT INTO sales (seller, day,
amount)
VALUES
('Alice', '2024-01-01', 100),
('Alice', '2024-01-02', 150),
('Bob',   '2024-01-01', 200),
('Bob',   '2024-01-02', 50);
```

```
SELECT * FROM sales;
```

1. Aggregation (SUM, COUNT, AVG…)

What is the total sales amount?

What is the total sales amount?

```
SELECT SUM(amount) AS total_sales
FROM sales;
```

Result

total_sales
500

Result- Aggregation

- One single number
- Rows are collapsed into one result

Aggregation with GROUP BY

How much did each seller sell in total?

How much did each seller sell in total?

```
SELECT seller, SUM(amount) AS total_sales
FROM sales
GROUP BY seller;
```

Result

seller	total_sales
Alice	250
Bob	250

2. Window function (aggregation without losing rows)

Show sale details (seller, day, amount) AND the total sales per seller at the same time.

Show each sale AND the total sales per seller at the same time.

```
SELECT  seller,  day,  amount,  SUM(amount) OVER (PARTITION BY seller) AS
total_sales_per_seller
FROM sales;
```

Result

seller	day	amount	total_sales_per_seller
Alice	2024-01-01	100	250
Alice	2024-01-02	150	250
Bob	2024-01-01	200	250
Bob	2024-01-02	50	250

Result- Window function

- No rows removed.
- Aggregation happens over a window.
- You keep row-level details and see the totals.

Use Aggregation -> When you want ONE global metric.

Total revenue? Average order value? Number of users?

Use GROUP BY -> When you want metrics per category AND don’t need row details. One row per group is enough.

Sales per seller? Orders per day? Revenue per country?

Use Window Functions->When you need aggregated insight while keeping row-level data.
You need both detail and summary or comparisons.

Show each sale + seller total.
Running total over time.
Rank users inside each group.
Compare row vs group average.
Compare across rows (past/future)

What is a Window Function in PostgreSQL?

A window function performs calculations across a set of rows related to the current row, similar to grouping and aggregate functions.

However, unlike aggregation, window functions do not combine rows into a single output row; each row remains separate.

How to write a window function?

Normal function OVER() = window function

Steps:

Step1: Write the function you already know:

SUM(amount)
(This would require GROUP BY.)

Step2: Add OVER() to that

SUM(amount) OVER ()

Steps:

Step3: Add meaning inside OVER()

SUM(amount) OVER (**PARTITION BY** seller)

It Calculates the sum per seller for each row

Step4: Add ORDER BY inside OVER()

SUM(amount) OVER (PARTITION BY seller **ORDER BY** day)

Window Function Syntax

```
function_name() OVER (
  PARTITION BY column_to_group_by
  ORDER BY column_to_order_by
)
```

- **function_name()**: like SUM(), AVG(), ROW_NUMBER(), etc.
- **OVER**: Tells SQL to apply the function as a window function.
- PARTITION BY (optional): groups rows (like grouping but keeps all rows).
- ORDER BY (optional): orders rows inside each group (needed for things like running totals or rankings).

For each sale, show the total sales amount across all sellers.

For each sale, show the total sales amount across all sellers.

```
SELECT  seller,  day,  amount,
SUM(amount) OVER () AS total_sales
FROM sales;
```

No PARTITION:
Total sales for the whole table repeated on every row.

For each sale, show the total sales amount across all sellers.

seller	day	amount	total_sales
Alice	2024-01-01	100	500
Alice	2024-01-02	150	500
Bob	2024-01-01	200	500
Bob	2024-01-02	50	500



For each sale, what is the total sales amount of the seller?

For each sale, what is the total sales amount of the seller?

```
SELECT  seller,  day,  amount,
 SUM(amount) OVER (PARTITION BY seller) AS
total_per_seller
FROM sales;
```

For each sale, what is the total sales amount of the seller?

seller	day	amount	total_per_seller
Alice	2024-01-01	100	250
Alice	2024-01-02	150	250
Bob	2024-01-01	200	250
Bob	2024-01-02	50	250



For each sale, how much has the seller sold up to that day?

For each sale, how much has the seller sold up to that day?

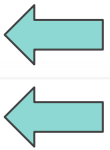
```
SELECT  seller,  day,  amount,

SUM(amount) OVER (PARTITION BY seller ORDER BY day) AS running_total

FROM sales;
```

For each sale, how much has the seller sold up to that day?

seller	day	amount	running_total
Alice	2024-01-01	100	100
Alice	2024-01-02	150	250
Bob	2024-01-01	200	200
Bob	2024-01-02	50	250



- Example: **ORDER BY price ASC** means window function will work on rows sorted by price inside each partition.

  Default sort order is ascending (ASC) if you don’t specify anything. (From smallest to largest or A to Z.)

- **NULLS FIRST | LAST (optional)**

Controls the position of NULL values when sorting.

- **NULLS FIRST:** Null values come before non-null values.
- **NULLS LAST**: Null values come after non-null values **(default)**.

```
INSERT INTO sales (seller, day, amount)
VALUES ('Alice', NULL, 100);


SELECT seller, day, amount, SUM(amount) OVER (
PARTITION BY seller ORDER BY day DESC NULLS FIRST
) AS total_sales_per_seller
FROM sales;
```

```
SELECT
  product_name,
  price,
  group_id,
  RANK() OVER (
    PARTITION BY group_id
    ORDER BY price DESC
  ) AS price_rank
FROM products;
```

It ranks products by price within each group, showing how they compare to each other without combining any rows.

Dataset: Products

ROW_NUMBER Example

```
SELECT product_name, group_name, price,
 ROW_NUMBER() OVER (PARTITION BY group_name ORDER BY price)
FROM products JOIN product_groups USING (group_id);
```

- Number each row starting from 1 within its group (1,2,3 laptop- 1,2,3 smartphone).

Datasets: Products and Product_groups

RANK vs DENSE_RANK

```
-- RANK
RANK() OVER (PARTITION BY group_name ORDER BY price)

-- DENSE_RANK
DENSE_RANK() OVER (PARTITION BY group_name ORDER
BY price)
```

- **RANK:** If two items have the same score, they get the same rank, but the next rank skips numbers.
  Example: 1, 1, 3, 4...

- **DENSE_RANK:** If two items have the same score, they get the same rank, and the next rank continues without skipping.
  Example: 1, 1, 2, 3...

Example_ Rank

```
SELECT
    product_id,
    product_name,
    group_name,
    price,
    RANK() OVER (
        PARTITION BY group_name
        ORDER BY price
    ) AS rank_in_group_by_price
FROM
    products
INNER JOIN
    product_groups
USING (group_id);
```

For each group_name (like Smartphone, Laptop, Tablet), it orders products by price (Ascending).

ORDER BY

- It sorts your results by a column.
- For example, it can show products from cheapest to most expensive.
- It just changes the order you see the rows.

RANK()

- It gives each row a rank number based on the order of a column.
- For example, the most expensive product gets rank 1, next gets 2, and so on.
- It adds a new column with the rank but doesn’t change the row order by itself.

Example- DENSE_RANK

```
SELECT
    product_id,
    product_name,
    group_name,
    price,
    DENSE_RANK() OVER (
        PARTITION BY group_name
        ORDER BY price
    ) AS dense_rank_in_group
FROM
    products
INNER JOIN
    product_groups
USING (group_id);
```

Multiple window functions

```
SELECT
  product_name,
  group_name,
  price,
  RANK() OVER (PARTITION BY group_name ORDER BY price DESC) AS
price_rank,
  ROW_NUMBER() OVER (PARTITION BY group_name ORDER BY price
DESC) AS row_num
FROM
  products
JOIN
  product_groups USING (group_id);
```